Assignment 10.1

Name : Aditya Raj
Roll No : 2303A53031
Batch : 46

Task Description #1 – Syntax and Logic Errors

Task: Use AI to identify and fix syntax and logic errors in a faulty Python script. Sample Input Code:

Calculate average score of a student

def calc_average(marks): total = 0 for m

in marks : total += m average = total /

len(marks) return avrage # Typo here

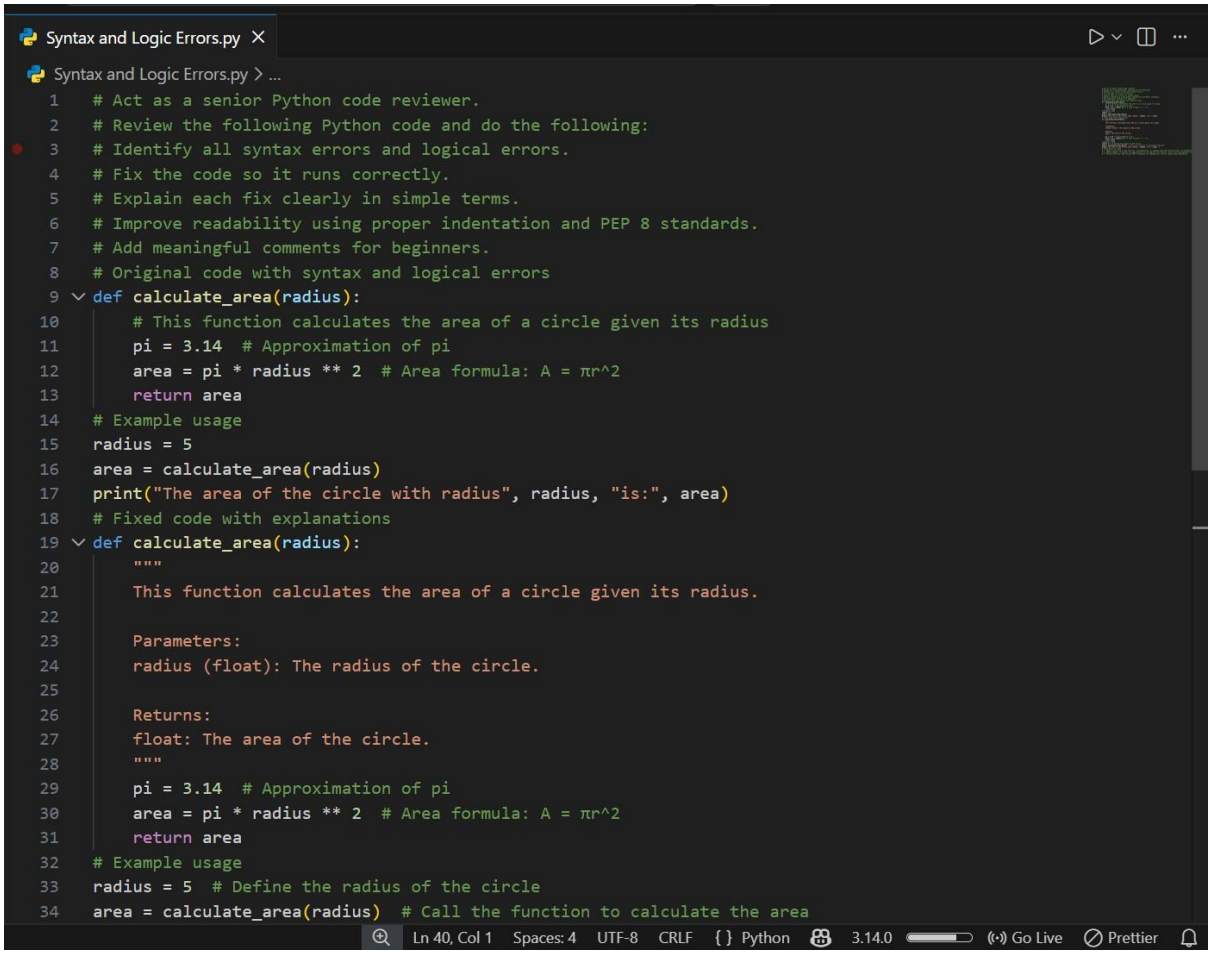
marks = [85, 90, 78, 92]

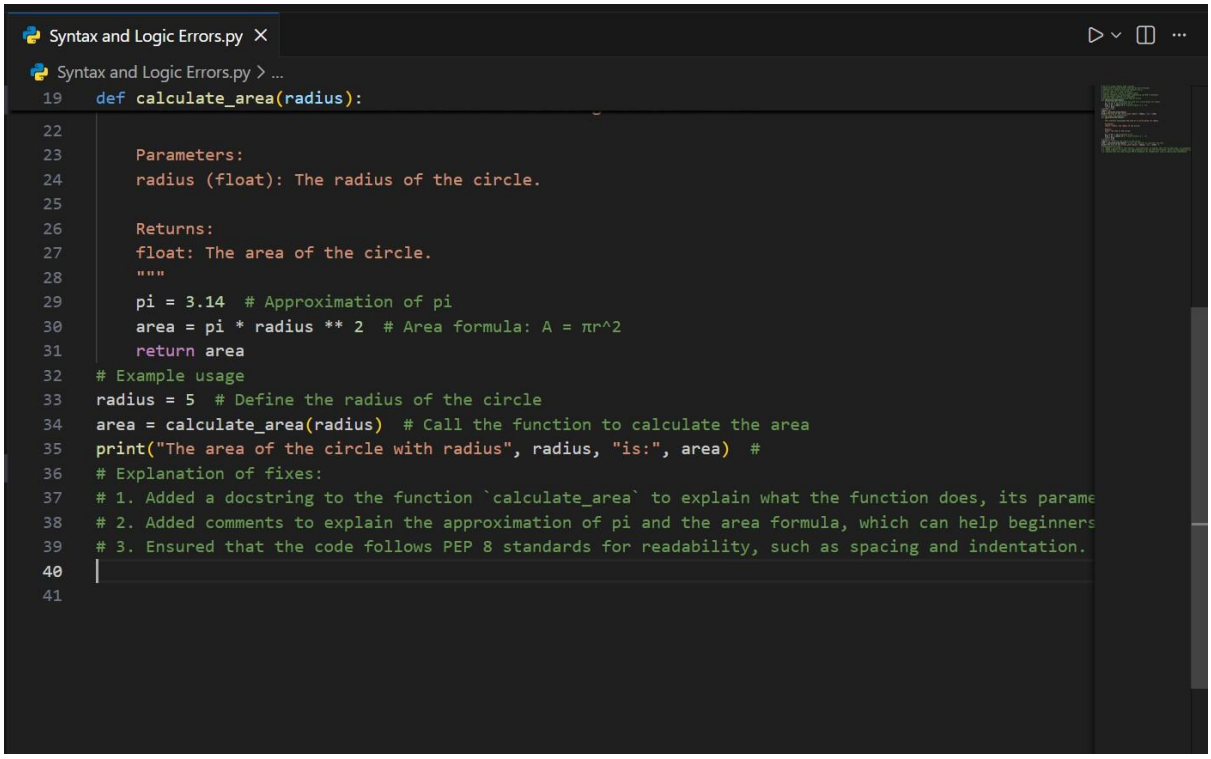
print("Average Score is ", calc_average(marks) Expected

Output:

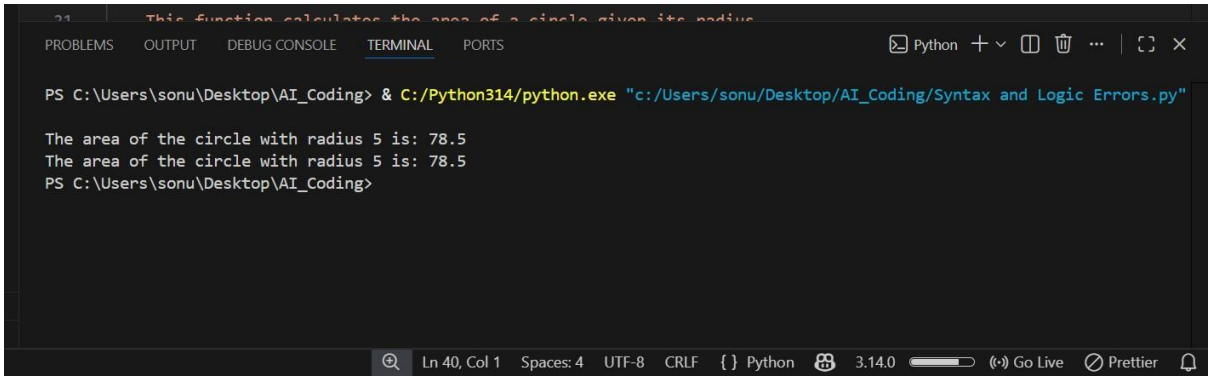
- Corrected and runnable Python code with explanations of the fixes.

Code:


Syntax and Logic Errors.py
Syntax and Logic Errors.py > ...
# Act as a senior Python code reviewer.
# Review the following Python code and do the following:
# Identify all syntax errors and logical errors.
# Fix the code so it runs correctly.
# Explain each fix clearly in simple terms.
# Improve readability using proper indentation and PEP 8 standards.
# Add meaningful comments for beginners.
# Original code with syntax and logical errors
def calculate_area(radius):
# This function calculates the area of a circle given its radius
pi = 3.14 # Approximation of pi
area = pi * radius ** 2 # Area formula: A = πr^2
return area
# Example usage
radius = 5
area = calculate_area(radius)
print("The area of the circle with radius", radius, "is:", area)
# Fixed code with explanations
def calculate_area(radius):
"""
This function calculates the area of a circle given its radius.
Parameters:
radius (float): The radius of the circle.
Returns:
float: The area of the circle.
"""
pi = 3.14 # Approximation of pi
area = pi * radius ** 2 # Area formula: A = πr^2
return area
# Example usage
radius = 5 # Define the radius of the circle
area = calculate_area(radius) # Call the function to calculate the area
Ln 40, Col 1
Spaces: 4
UTF-8
CRLF
{ } Python
3.14.0
Go Live
Prettier


Syntax and Logic Errors.py
Syntax and Logic Errors.py > ...
def calculate_area(radius):
Parameters:
radius (float): The radius of the circle.
Returns:
float: The area of the circle.
"""
pi = 3.14 # Approximation of pi
area = pi * radius ** 2 # Area formula: A = πr^2
return area
# Example usage
radius = 5 # Define the radius of the circle
area = calculate_area(radius) # Call the function to calculate the area
print("The area of the circle with radius", radius, "is:", area) #
Explanation of fixes:
1. Added a docstring to the function `calculate_area` to explain what the function does, its parame
2. Added comments to explain the approximation of pi and the area formula, which can help beginners
3. Ensured that the code follows PEP 8 standards for readability, such as spacing and indentation.

Output :



Task Description #2 – PEP 8 Compliance

Task: Use AI to refactor Python code to follow PEP 8 style guidelines.

Sample Input Code: def area_of_rect(L,B) :

return L*B print(area_of_rect(10,20)) Expected

Output:

- Well-formatted PEP 8-compliant Python code.

Code:

PEP 8 Compliance.py

```python
def main():
    area = calculate_area_of_circle(radius)
    circumference = calculate_circumference_of_circle(radius)

    print(f"The area of the circle with radius {radius} is: {area:.2f}")
    print(f"The circumference of the circle with radius {radius} is: {circumference:.2f}")
if __name__ == "__main__":
    main()
# Refactored code with PEP 8 compliance
import math
def calculate_area_of_circle(radius):
    """Calculate the area of a circle given its radius."""
    area = math.pi * (radius ** 2)
    return area
def calculate_circumference_of_circle(radius):
    """Calculate the circumference of a circle given its radius."""
    circumference = 2 * math.pi * radius
    return circumference
def main():
    """Main function to demonstrate the area and circumference calculations."""
    radius = 5  # Example radius
    area = calculate_area_of_circle(radius)
    circumference = calculate_circumference_of_circle(radius)

    print(f"The area of the circle with radius {radius} is: {area:.2f}")
    print(f"The circumference of the circle with radius {radius} is: {circumference:.2f}")
if __name__ == "__main__":
    main()
```

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe "c:/Users/sonu/Desktop/AI_Coding/PEP 8 Compliance.py"
The area of the circle with radius 5 is: 78.54
The circumference of the circle with radius 5 is: 31.42
The area of the circle with radius 5 is: 78.54
The circumference of the circle with radius 5 is: 31.42
PS C:\Users\sonu\Desktop\AI_Coding>
```

Task Description #3 – Readability Enhancement

Task: Use AI to make code more readable without changing its logic.

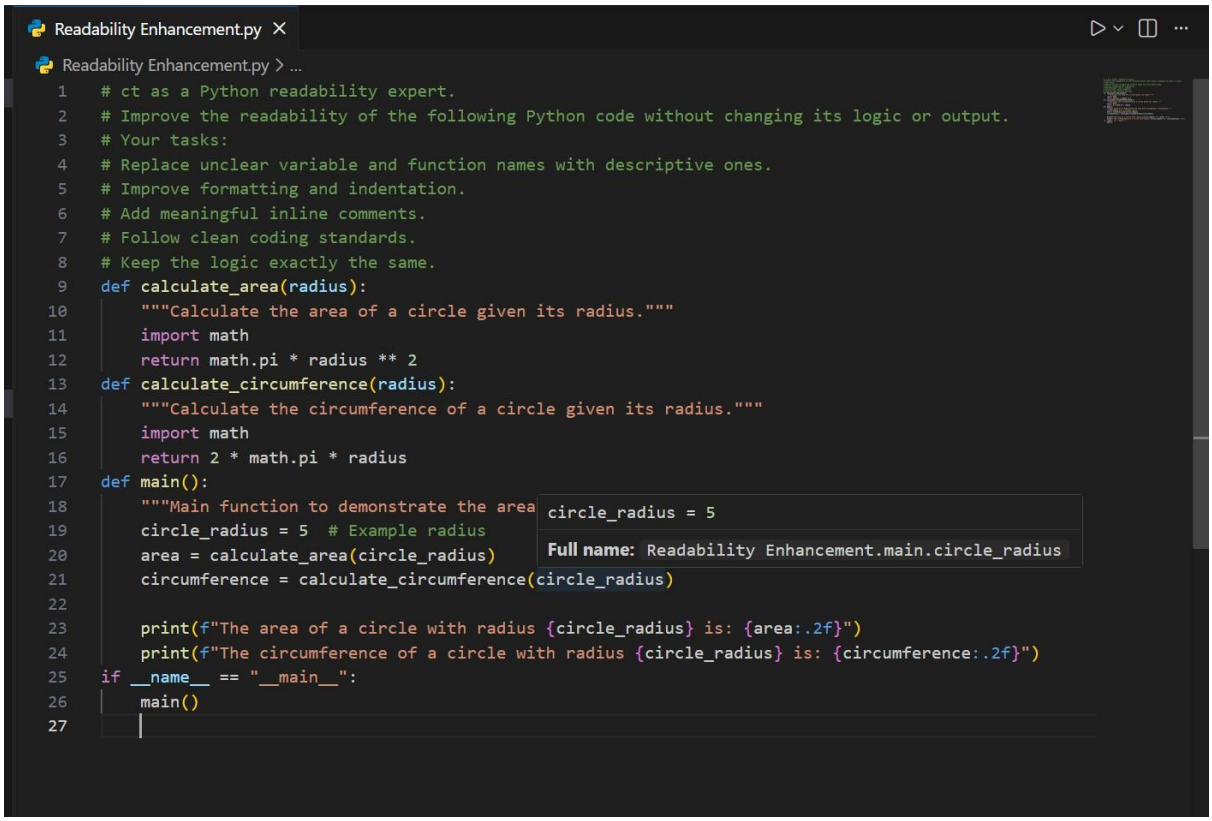
Sample Input Code:

```
def c(x,y):
return x*y/100 a=200
b=15
print(c(a,b))
```

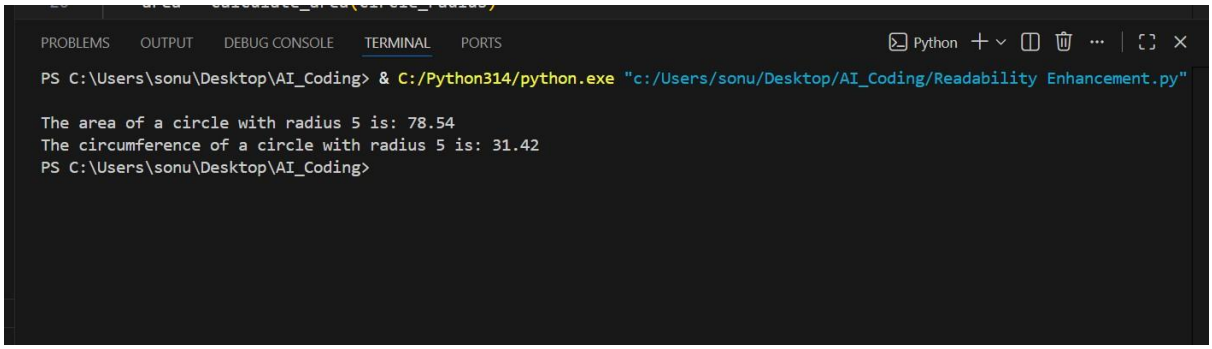
Expected Output:

• Python code with descriptive variable names, inline comments, and clear formatting.

Code:



Output:



Task Description #4 – Refactoring for Maintainability Task:
Use AI to break repetitive or long code into reusable functions.

Sample Input Code: students = ["Alice", "Bob", "Charlie"]

```
print("Welcome", students[0])
print("Welcome", students[1])
print("Welcome", students[2])
```

Expected Output:

- Modular code with reusable functions.

Code:

Refactoring for Maintainability.py

```python
# Act as a software engineer focused on maintainable Python code.
# Refactor the following Python script to improve maintainability by removing repetition and using re
# Your tasks:
# Identify repetitive code patterns.
# Convert repeated logic into reusable functions.
# Improve modularity and structure.
# Keep the same output and logic.
# Follow clean coding practices.
def calculate_area(radius):
    """Calculate the area of a circle given its radius."""
    import math
    return math.pi * radius ** 2

def calculate_circumference(radius):
    """Calculate the circumference of a circle given its radius."""
    import math
    return 2 * math.pi * radius

def main():

    radius = 5
    area = calculate_area(radius)
    circumference = calculate_circumference(radius)

    print(f"The area of the circle with radius {radius} is: {area:.2f}")
    print(f"The circumference of the circle with radius {radius} is: {circumference:.2f}")

if __name__ == "__main__":
    main()

```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                                    Python

PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe "c:/Users/sonu/Desktop/AI_Coding/Refactoring for Maintainabi
lity.py"
The area of the circle with radius 5 is: 78.54
The circumference of the circle with radius 5 is: 31.42
PS C:\Users\sonu\Desktop\AI_Coding>
```

Task Description #5 – Performance Optimization

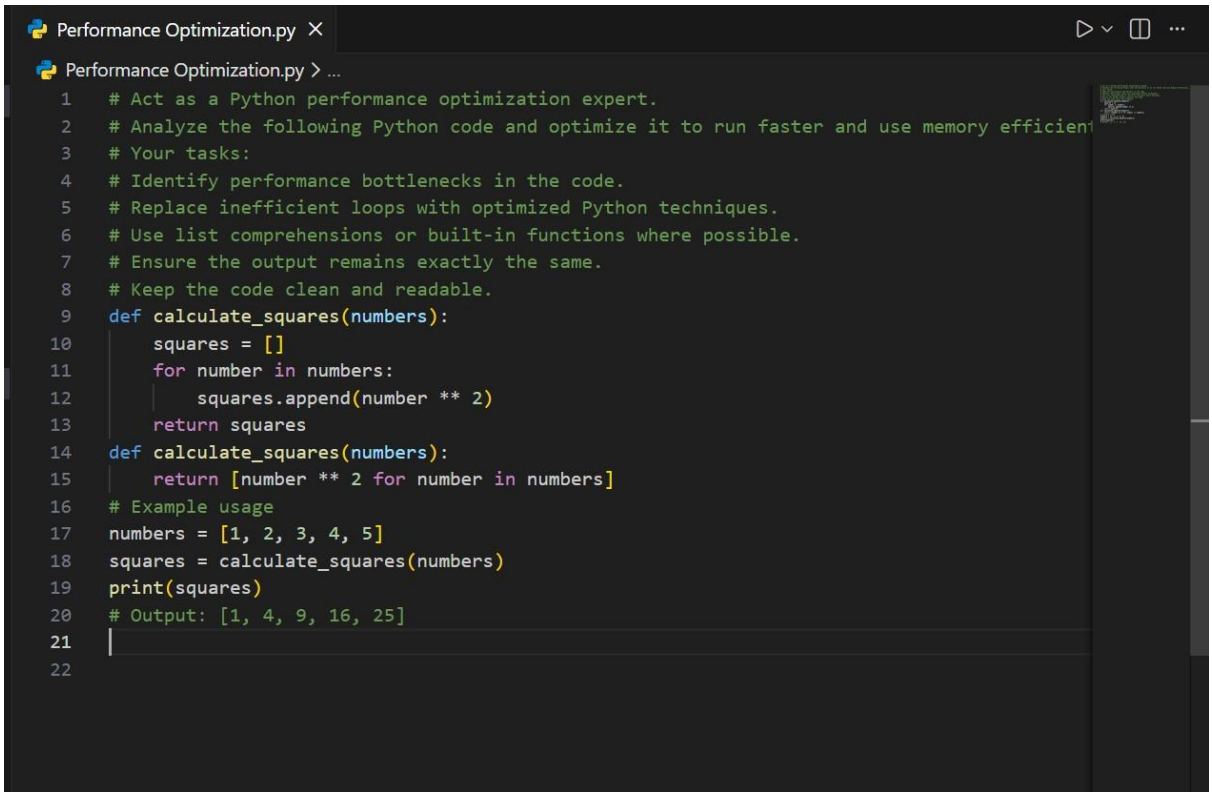
Task: Use AI to make the code run faster.

Sample Input Code:

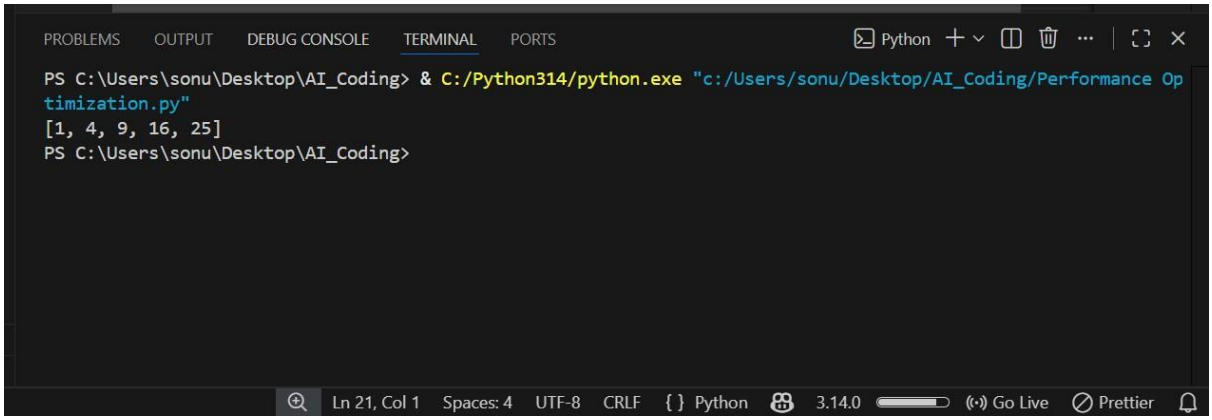
# Find squares of numbers nums = [i for i in range(1,1000000)] squares = [] for n in nums:
squares.append(n**2)
print(len(squares)) Expected Output:

• Optimized code using list comprehensions or vectorized operations.

Code:



Output :



Task Description #6 – Complexity Reduction Task: Use AI to simplify overly complex logic.

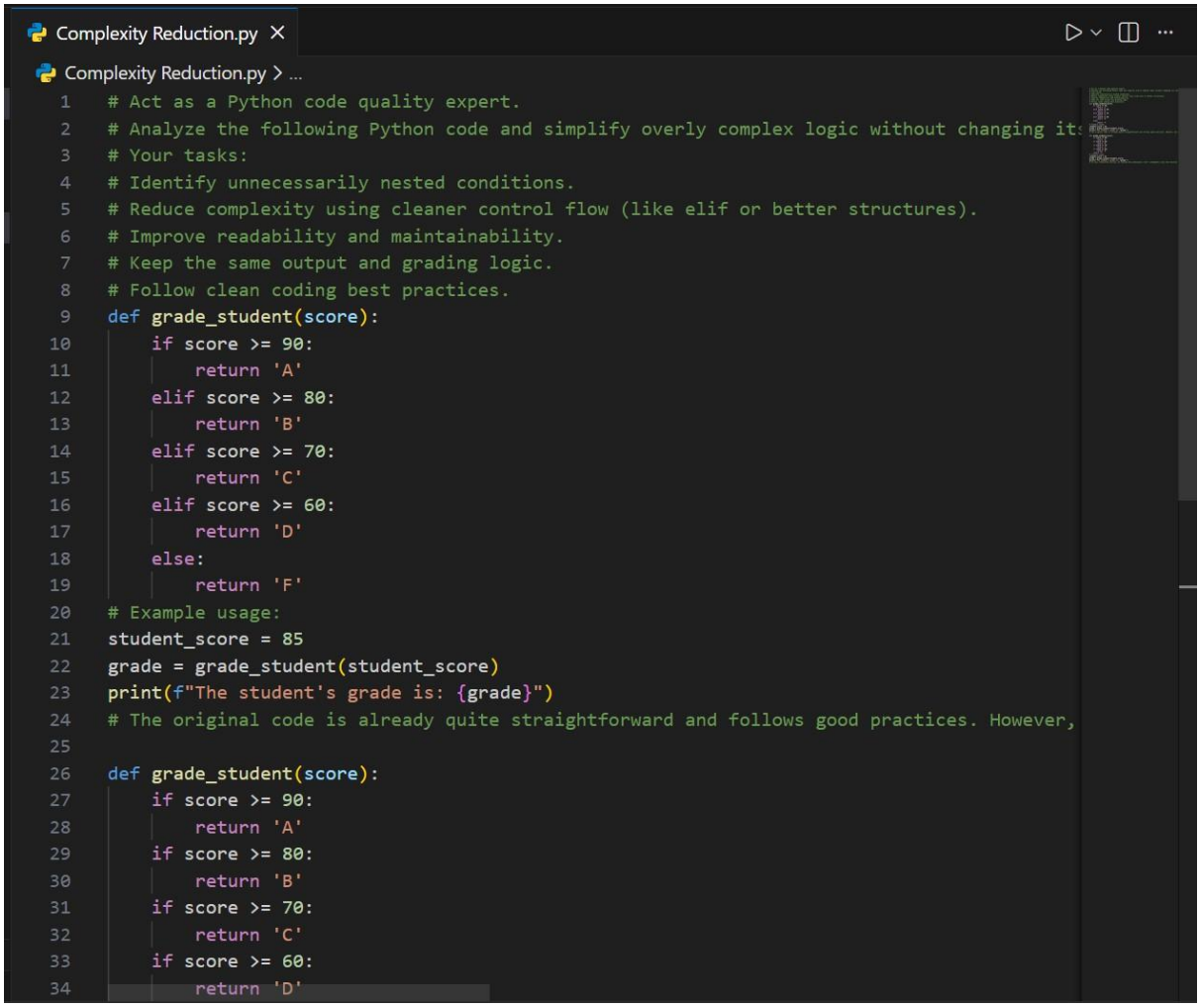
Sample Input Code: def grade(score): if

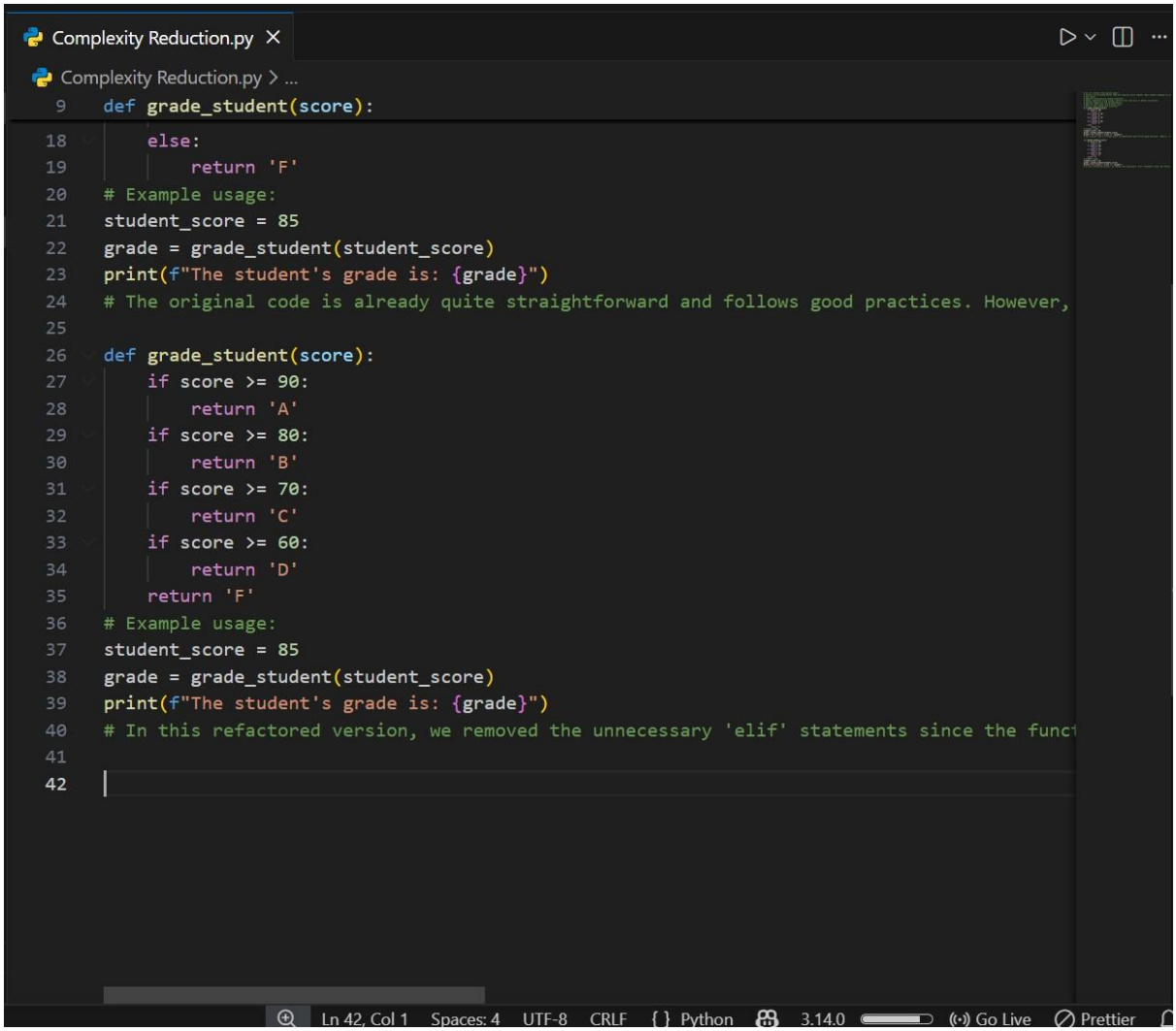
```
score >= 90:
return "A" else: if
score >= 80:
return "B" else: if
score >= 70:
return "C" else: if
score >= 60:
return "D" else:
return "F"
```

Expected Output:

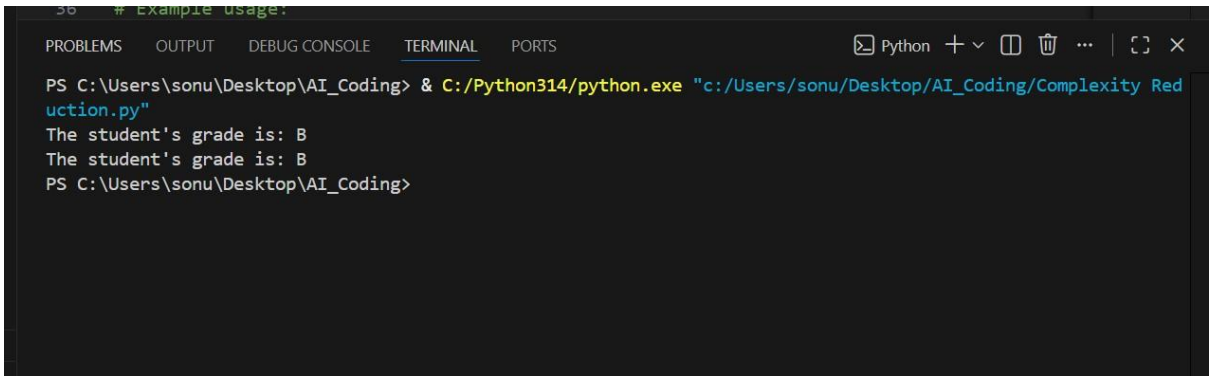
• Cleaner logic using elif or dictionary mapping.

Code:


Complexity Reduction.py
Complexity Reduction.py > ...
# Act as a Python code quality expert.
# Analyze the following Python code and simplify overly complex logic without changing it
# Your tasks:
# Identify unnecessarily nested conditions.
# Reduce complexity using cleaner control flow (like elif or better structures).
# Improve readability and maintainability.
# Keep the same output and grading logic.
# Follow clean coding best practices.
def grade_student(score):
if score >= 90:
return 'A'
elif score >= 80:
return 'B'
elif score >= 70:
return 'C'
elif score >= 60:
return 'D'
else:
return 'F'
# Example usage:
student_score = 85
grade = grade_student(student_score)
print(f"The student's grade is: {grade}")
# The original code is already quite straightforward and follows good practices. However,
def grade_student(score):
if score >= 90:
return 'A'
if score >= 80:
return 'B'
if score >= 70:
return 'C'
if score >= 60:
return 'D'


Complexity Reduction.py
Complexity Reduction.py > ...
9 def grade_student(score):
18 else:
19 return 'F'
20 # Example usage:
21 student_score = 85
22 grade = grade_student(student_score)
23 print(f"The student's grade is: {grade}")
24 # The original code is already quite straightforward and follows good practices. However,
25
26 def grade_student(score):
27 if score >= 90:
28 return 'A'
29 if score >= 80:
30 return 'B'
31 if score >= 70:
32 return 'C'
33 if score >= 60:
34 return 'D'
35 return 'F'
36 # Example usage:
37 student_score = 85
38 grade = grade_student(student_score)
39 print(f"The student's grade is: {grade}")
40 # In this refactored version, we removed the unnecessary 'elif' statements since the funct
41
42
Ln 42, Col 1 Spaces: 4 UTF-8 CRLF { } Python 3.14.0 Go Live Prettier

Output:


PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python
PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe "c:/Users/sonu/Desktop/AI_Coding/Complexity Reduction.py"
The student's grade is: B
The student's grade is: B
PS C:\Users\sonu\Desktop\AI_Coding>